Exercise 1: Let C1 and C2 be two consistent cuts. Show that the intersection of C1 and C2 is a consistent cut.

def A cut C is consistent iff: $e \longrightarrow e' \wedge e' \in C \Rightarrow e \in C$

Let $e' \in C = C_1 \cap C_2$ and e t.c. $e \longrightarrow e'$.

By definition, since $e' \in C_1 \cap C_2$ we also have $e' \in C_1$ and $e' \in C_2$.

Howevevre, since $e \longrightarrow e'$ and C_1 is a consistent cut, e also MUST be in C_1.
Analog reasoning for C_2. Because $e \longrightarrow e'$ and C_2 is consistent, $e \in C_2$.
At this point, $e \in C_1 \wedge e \in C_2 \Rightarrow e \in C = C_1 \cap C_2$

So, $\forall e, e' \quad e' \in C = C_1 \cap C_2 \wedge e \longrightarrow e' \Rightarrow e \in C_1 \cap C_2 \Rightarrow C_1 \cap C_2$ CONSISTENT

Exercise 2: Let C1 and C2 be two consistent cuts. Show that the union of C1 and C2 is a consistent cut.

def A cut C is CONSISTENT iff: $e' \in C \wedge e \longrightarrow e' \Rightarrow e \in C$

Let $e' \in C = C_1 \cup C_2$ and e t.c. $e \longrightarrow e'$.

By definition of "consistent cut", since $e' \in C_1$ and $e \longrightarrow e'$, we have that also $e \in C_1$.
And $e \in C_1 \Rightarrow e \in C = C_1 \cup C_2$. (Same for C_2)

So, $\forall e' \in C = C_1 \cup C_2 \wedge e \longrightarrow e' \Rightarrow e \in C = C_1 \cup C_2 \Rightarrow C_1 \cup C_2$ CONSISTENT

Exercise 3: Show that every consistent global state can be reached by some consistent run.

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

Then we can apply this sorting algorithm to C and $E \setminus C$, in sequence, to create a run R:

$R = sort(C)$ followed by $sort(E \setminus C)$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if $e \in C$ and $e' \in C$ then e will be placed before e' (by definition of $sort(\cdot)$)
- if $e \in C$ and $e' \in E \setminus C$ then e will be placed before e', since e will be in $sort(C)$ and e' will be in $sort(E \setminus C)$
- if $e \in E \setminus C$ and $e' \in C$ then C would not be consistent. So, this case is not possible
- if $e \in E \setminus C$ and $e' \in E \setminus C$ then e will be placed before e' (by definition of $sort(\cdot)$)

Exercise 4: Let C1 and C2 be two consistent cuts. If C1 is a subset of C2, then C2 is reachable from C1. (There exists a consistent run that reaches C1 and then reaches C2.)

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

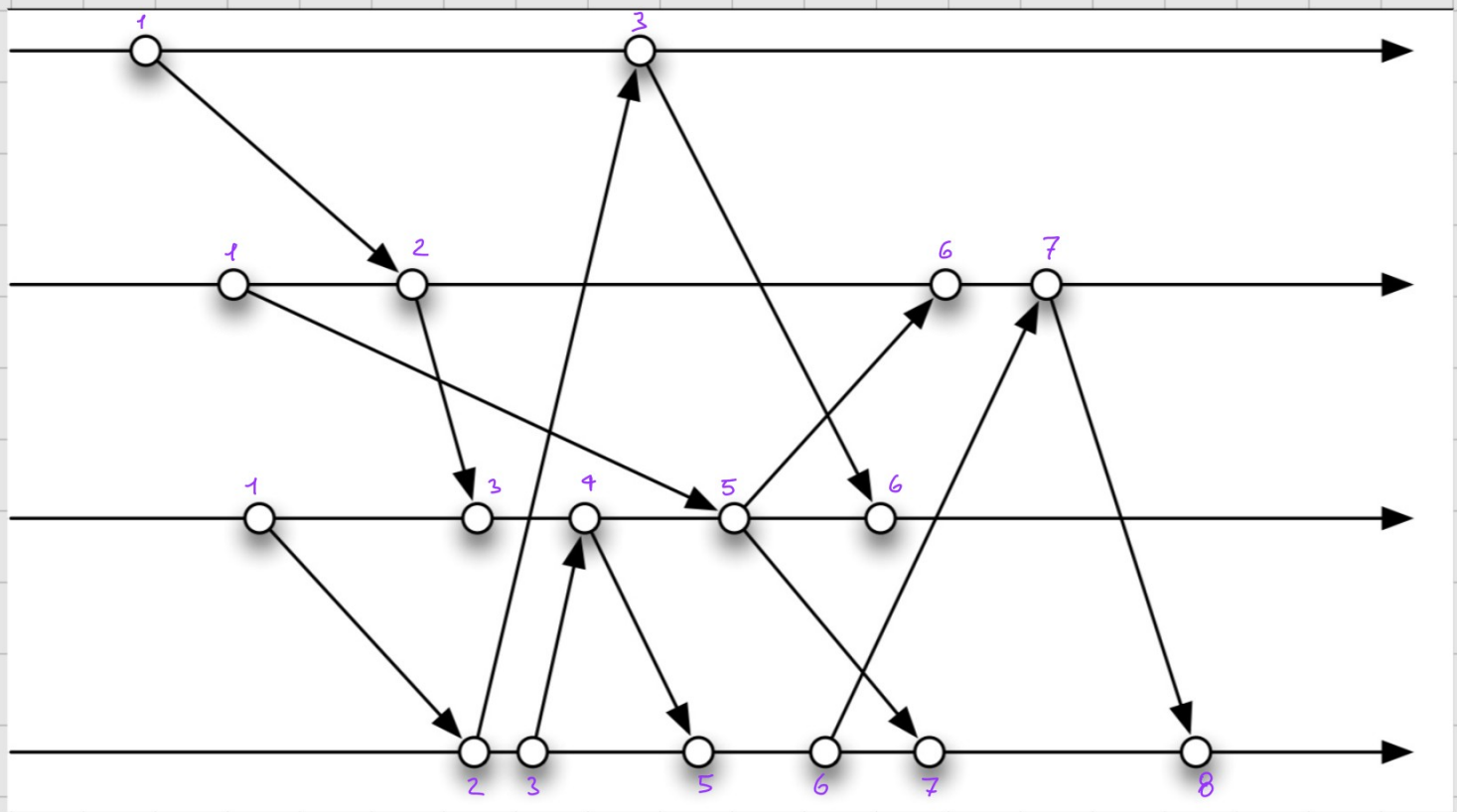
Since $C_1 \subseteq C_2$ we can divide E into three disjoint sets: C_1, $C_2 \setminus C_1$ and $E \setminus C_2$. Then we can apply this sorting algorithm to C_1, $C_2 \setminus C_1$ and $E \setminus C_2$, in sequence, to create a run R:

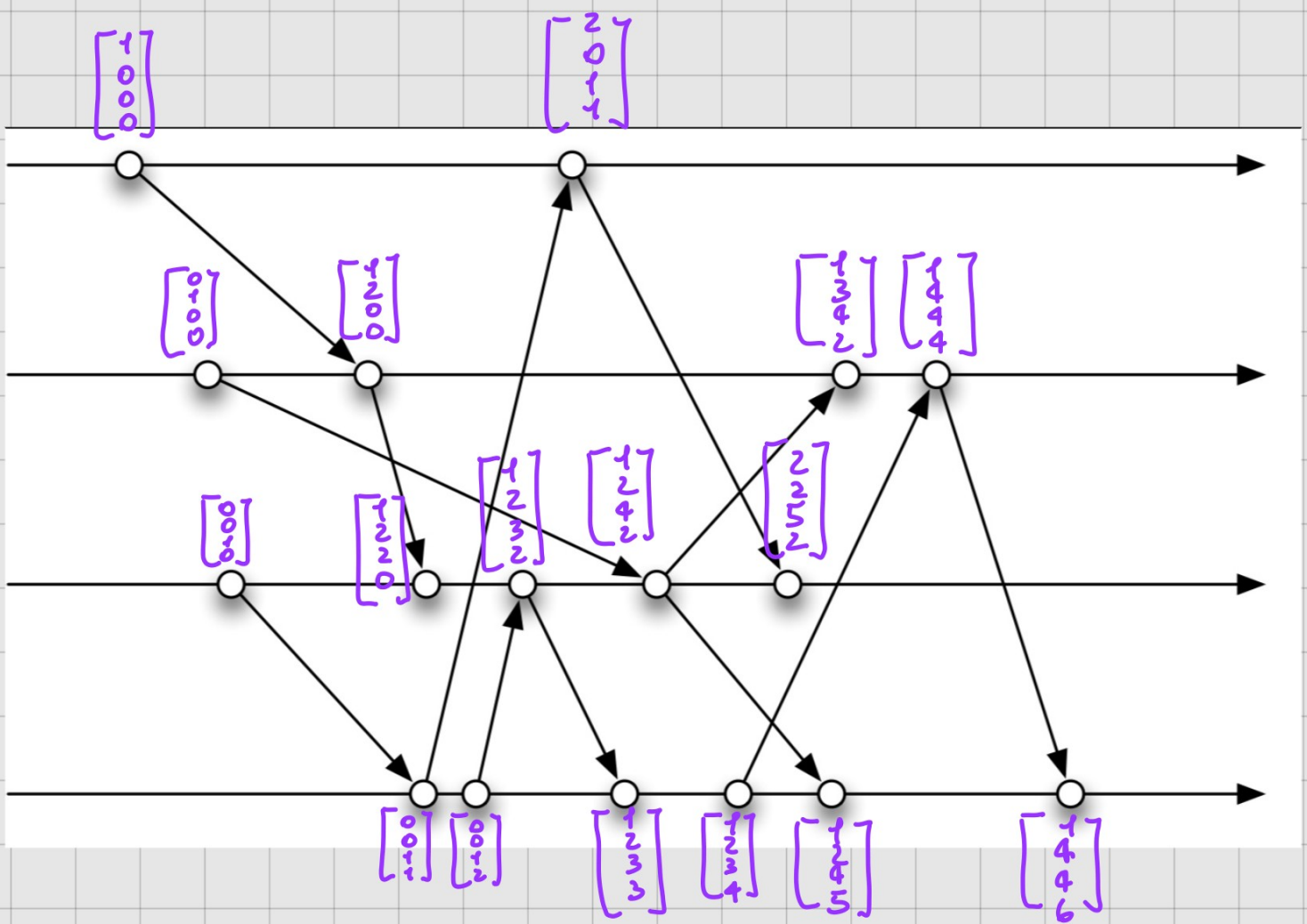
$$R = sort(C_1) \text{ followed by } sort(C_2 \setminus C_1) \text{ followed by } sort(E \setminus C_2)$$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if e and e' are in the same set (both in C_1, both in $C_2 \setminus C_1$ or both in $E \setminus C_2$) then e will be placed before e' (by definition of $sort(\cdot)$)
- if e and e' are in different sets "left to right" ($e \in C_1 \wedge e' \in C_2 \setminus C_1 \cup E \setminus C_1$ or $e \in C_2 \setminus C_1 \wedge e' \in E \setminus C_2$) then e will be placed before e' by construction of R ($sort(C_1)$ appears before $sort(C_2 \setminus C_1)$ that appears before $sort(E \setminus C_2)$)
- if e and e' are in different sets "right to left", C_1 or C_2 would not be consistent. So this case is not possible.

Exercise 5: Label with proper logical clock all the events of the distributed computation in image vector.pdf. (You can consider events that receive a message and immediately send it as single events.)





Exercise 6: Show that the history of an event is a consistent cut.

Let's proceed by CONTRADICTION. Let's assume that $H(e')$ is NOT consistent. This means that:

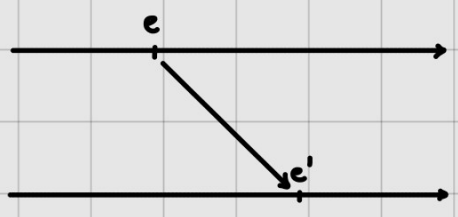
$$\exists\, x, y \text{ such that } y \longrightarrow x \text{ and } x \in H(e') \text{ BUT } y \notin H(e')$$

However, since $x \in H(e')$ we have that $x \longrightarrow e'$. At this point, applying the transitivity, $y \longrightarrow x$ and $x \longrightarrow e \Rightarrow y \longrightarrow e \Rightarrow y \in H(e) \Rightarrow$ CONTRADICTION.

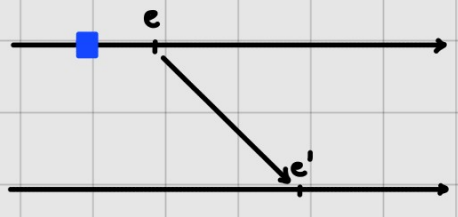
Exercise 7: Show that the Chandy-Lamport Snapshot Protocol builds a consistent global state.

Dim. To prove that the Chandy-Lamport algorithm always generates a consistent cut we can use a proof by contradiction

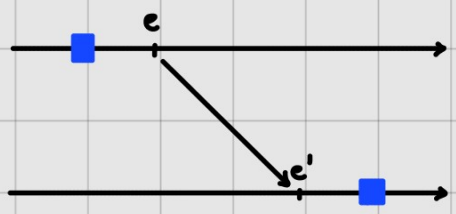
Let's assume that C is the cut generated from the Chandy-Lamport protocol, and there are 2 events e, e' such that $e \longrightarrow e'$, $e' \in C$ BUT $e \notin C$.



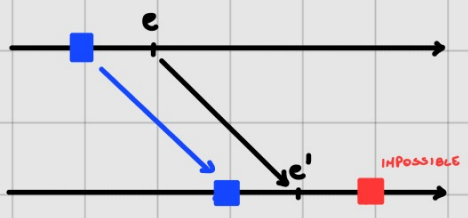
Since $e \notin C$, the "snapshot message" must have arrived before e



And, since $e' \in C$, the "snapshot message" must have arrived after e'

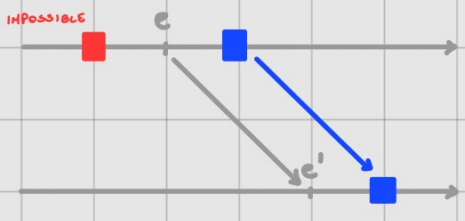


However, since the Chandy-Lamport Clock Protocol works on the assumption of a FIFO CHANNEL, the "snapshot message" send before e MUST arrive before e'



Consequently, both $e, e' \notin C \Rightarrow C$ is consistent $\Rightarrow$ CONTRADICTION (we also have a contradiction on the premise that $e' \in C$)

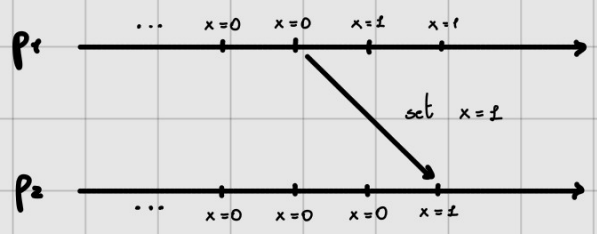
We can also apply the same logic on the "snapshot message" arrived after e'. Since the channel is FIFO it must have been sent after e.



Consequently, both $e, e' \in C \Rightarrow C$ is consistent $\Rightarrow$ CONTRADICTION (we also have a contradiction on the premise that $e \notin C$)

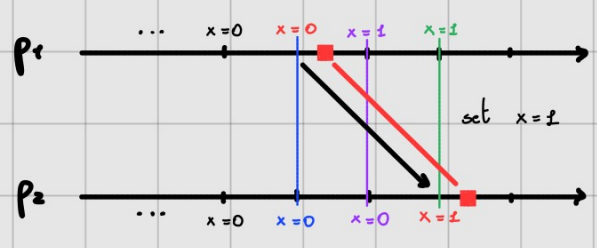
Exercise 8: Show that the Chandy-Lamport Snapshot Protocol can build a global state that never happened.

In this case it is sufficient to find a counter-example. Let's imagine a system with 2 processes, both with variable x initialized as x = 0. Then, process p_1 sends a message: "set x = 1" to p_2. Then, p_1 sets its x = 1.



At this point:

- The real global states happened are (0, 0), (1, 0) and (1, 1).
- The Chandy-Lamport protocol, instead, captures the state (0, 1) that never really happened



Exercise 9: What good is a distributed snapshot when the system was never in the state represented by the distributed snapshot? Give an application of distributed snapshots.

Even if a distributed snapshot might not represent a real execution of the system, it is still consistent meaning that it is equivalent to a real one.

The real execution goes from the initial state S_0 to the final state S_f, and the snapshot S^* is reachable from S_0 and reaches S_f.

A real application could be deadlock detection or recovery from a system crash (all the nodes reload the same consistent global state sure that everything is ok even if that state could have never happened)

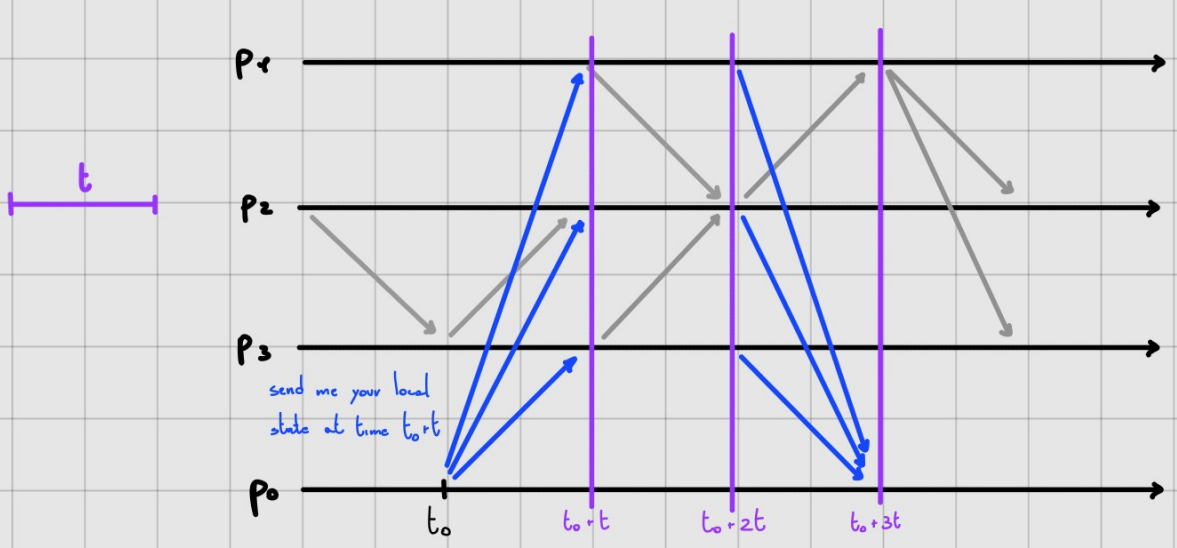
Exercise 10: Consider a distributed system where every node has its physical clock and all physical clocks are perfectly synchronized. Give an algorithm to record global state assuming the communication network is reliable. (Note that your algorithm should be simpler than the Chandy–Lamport algorithm.)

Let's assume that:

- t is the upper bound of time needed to deliver a message
- each message includes the exact timestamp in which it was sent

Protocol:

- At time t_0 a coordinator process p_0 broadcasts a message to all the other processes saying: "send me your local status at time $t_0 + t$".
- At time $t_0 + t$ all processes have received the message from p_0 and prepare their local state. However, since there could be messages that are still arriving, each process waits t time. If it receives a message with timestamp $< t_0 + t$ then it will be included in the local state.
- At time $t_0 + t + t$ we are sure that no other messages sent before $t_0 + t$ will arrive. So, each process finally sends it local state to p_0
- At time $t_0 + t + t$ p_0 will have received all the local states and can combine them to get the global state of the system at time $t_0 + t$



Exercise 11: What modifications should be done to the Chandy–Lamport snapshot protocol so that it records a strongly consistent snapshot (i.e., all channel states are recorded empty).

Each process p_i stops sending messages AS SOON AS it receives a "snapshot" message (marker). Then, p_i sends broadcasts its marker to all the other processes (as expected by the normal Chandy-Lamport Protocol).

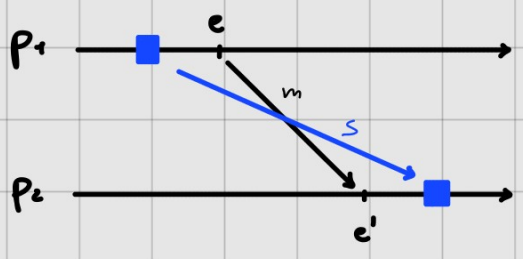
At this point, since the channel is FIFO and the marker is the last message sent by p_i, any other process p_j that receives that marker can be sure there are no other messages coming from p_i.

Applying this reasoning to all the processes, when a process receives all the markers from the others, it can be sure there are no messages in transit. So, it can record its local state. By combining all the local states we obtain the strongly consistent snapshot.

Exercise 12: Show that, if channels are not FIFO, then Chandy–Lamport snapshot algorithm does not work.

If the channel is NOT FIFO something like this could happen:

- the process p_1 takes the snapshot of its local state. Then sends the "snapshot request" s to the process p_2
- Immediatly after p_1 send a message m to p_2 (let's call this event e)
- Since the channel is NOT FIFO the process p_2 receives the message m before the marker s (let's call this event e').
- Consequently, the event e is NOT in the snapshot of p_1 but the event e' is in the snapshot of p_2. But $e \longrightarrow e'$... So, the final cut will not be consistent



Exercise 13: Let S0 be the global state when the Chandy-Lamport snapshot protocol starts, S be the global state built by the protocol, and S1 be the global state when the protocol ends. Show that S is reachable from S0 and that S1 is reachable from S. Remember that S might not have happened. (In other words, there is a consistent run that goes through S0, S1 and S.)

Due to the passage of time in the system we can obviously assume that $S_0 \subseteq S \subseteq S_f$.

At this point, the exercise becomes identical to Exercise4

Let E be the set of all events in the system. Exploiting the "happened before" relationship ($\longrightarrow$) we can create a sorting algorithm $sort(\cdot)$ such that:

- if $e \longrightarrow e'$ then e is placed before e'

And apply this sorting algorithm to S_0, $S \setminus S_0$ and $S \setminus S_f$, in sequence, to create a run R:

concatenate concatenate

$$R = sort(S_0) \mid sort(S_0 \setminus S) \mid sort(S \setminus S_f)$$

At this point, R is also consistent because $\forall e, e' \in E$ t.c. $e \longrightarrow e'$:

- if e and e' are in the same set (both in S_0, both in $S \setminus S_0$ or both in $S \setminus S_f$) then e will be placed before e' by definition of $sort(\cdot)$
- if e and e' are in different sets "left to right" ($e \in S_0 \wedge e' \in S_0 \setminus S \cup S \setminus S_f$ or $e \in S_0 \setminus S \wedge e' \in S \setminus S_f$) then e will be placed before e' by construction of R ($sort(C_f)$)
- if e and e' are in different sets "right to left", one of the sets would not be consistent. However, the Chandy-Lamport Protocol always generate consistent cuts... So this case is not possible

Exercise 14: Exercise 7.2 in file atomic_commit.pdf.

7.2 Give an ACP (→ ATOMIC COMMIT PROTOCOL) that also satisfies the converse of condition AC3. That is, if all processes vote Yes, then the decision must be Commit. Why is it not a good idea to enforce this condition?

Protocol:

1) The coordinator sends the VOTE REQUEST to each node

2) Each node sends its vote (YES or NO) to the coordinator

3) The coordinator waits forever until it receives all the votes
   - if all the votes are "YES" then COMMIT
   - if even a single vote is "NO" then ABORT

Clearly, this is not a good idea because we could wait forever without taking any decision.

AC1: The coordinator sends the same decision to everyone. So, every node will take the same decision of the others.

AC2: Assuming Log-then-Send for both coordinator and voters, if any node crashes, after the reboot it can simply read the log and take the same decision/vote as before

AC3: The only way to commit is that every node votes YES. So AC3 is preserved

AC4/AC5: Since the coordinator waits forever, whether there are failures or not, if all votes are YES it will decide COMMIT

Exercise 15: Exercise 7.3 in file atomic_commit.pdf.

7.3 Consider 2PC (2 Phase Commit) with the cooperative termination protocol. Describe a scenario (a particular execution) involving site failures only, which causes operational sites to become blocked.

There are 2 nodes and 1 coordinator:

- Both nodes vote YES and send the vote to the coordinator
- The coordinator receives the votes, then crashes
- Both nodes are now blocked waiting for the coordinator's decision. Even cooperating they cannot move on because they all voted yes.

Exercise 16: Show that Paxos is not live.

Since the proposers can start a new round in every moment, it could happen that every time the system is about to choose a value in round r_1, a different proposer starts a new round r_2, with $r_2 > r_1$.

In this scenario, if the prepare(r_2) messages arrive before the accept(r_1, x) ones, the acceptors will promise to participate in round r_2, ignoring the incoming accept(r_1, x) messages. Since this situation could repeat indefinitely the protocol could never end. Consequently, Paxos does not guarantee liveness.

Exercise 17: Assume that acceptors do not change their vote. In other words, if they vote for value v in round i, they will not send learn messages with value different from v in larger rounds. Show that Paxos, with this modification, is safe. Unfortunately, the modification introduces a severe liveness problem (the protocol can reach a livelock).

Let's proceed by contradiction. Let's assume that the system is not safe, so, that 2 different values v and w are choosen. However, to have a majority on v and a majority on w in the same round, at least one acceptor should have voted for both v and w. But the premises states that the acceptors cannot change their vote ⇒ contradiction ⇒ the system is safe.

About the liveness: as soon as no value can no longer reach a majority, the system is permanently blocked, since no acceptor can change its vote.

Exercise 18: How many messages are used in Paxos if no message is lost and in the best case? Is it possible to reduce the number of messages without losing tolerance to failures and without changing the number of proposers, acceptors, and learners?

In Classical Paxos, assuming that nodes do not send messages to themselves, we need at least:

- $(n-1)$ prepare messages (proposer ⟶ acceptors)
- $(n-1)$ promise messages (acceptors ⟶ proposer)
- $(n-1)$ accept messages (proposer ⟶ acceptors)
- $(n-1)n$ learn messages (because each of n nodes must send the learn to all the other $n-1$ nodes)

$$\text{Total} = 3(n-1) + (n-1)n = 3n - 3 + n^2 - n = n^2 + 2n - 3$$

Yes, it is possible. For example, sending the learn messages from the acceptors to a single proposer who will later send the final learn to all the other learners.

- $(n-1)$ prepare messages (proposer $\longrightarrow$ acceptors)
- $(n-1)$ promise messages (acceptors $\longrightarrow$ proposer)
- $(n-1)$ accept messages (proposer $\longrightarrow$ acceptors)
- $(n-1)$ learn messages (acceptors $\longrightarrow$ proposer)
- $(n-1)$ learn messages (proposer $\longrightarrow$ learners)

Total = $5(n-1)$

Now, it is not possible to reduce it further without losing fault tolerance (e.g., we could send messages only to the quorum, but in that case the system can't tolerate any failure).

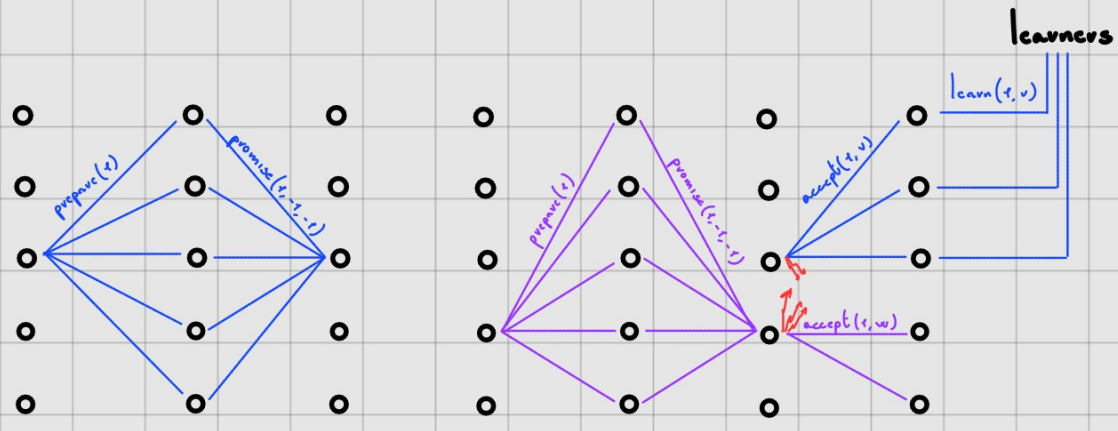
Exercise 19: Assume that you remove the property that every round is associate to a unique proposer. After collecting a quorum of n-f promises (where n is the number of acceptors and f is such that n=2f+1), the proposer chooses one of the values voted in max round in the promises (of course it is not unique, the proposer chooses just one in an arbitrary way). Show that Paxos is not safe any more.

Let's suppose a system with 5 nodes. Since we are in Classical Paxos the quorum is $Q = \lfloor \frac{5}{2} \rfloor + 1 = 3$.

- proposer p3 starts round 1 and all the acceptors reply with promises $(1, -1, -1)$ since they never voted before
- Immediately after, a different proposer p4 start another round 1 and the acceptors reply again with promises $(1, -1, -1)$ since they never voted before

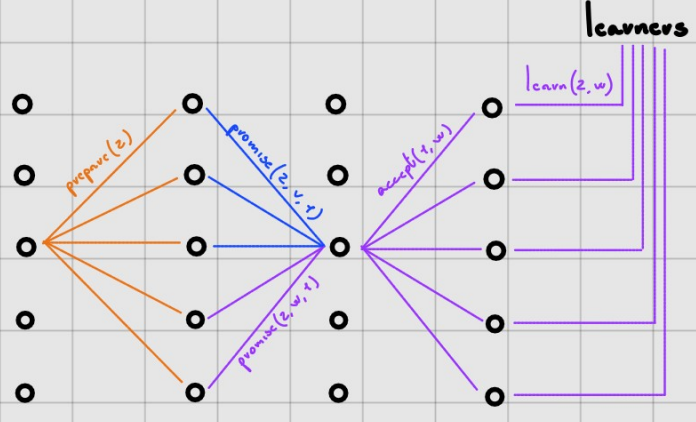
- At this point, p3 selects a random value v and p4 a random value w to be accepted.

- Some accept messages get lost, so 3 acceptors vote for v and 2 for w. So v is learned in round 1



- proposer p3 starts round 2. Since the acceptors already voted, there will be 3 promise(2, v, 1) and 2 promise(2, w, 1). So, the proposer p3 chooses w arbitrary and sends the accept(2, w) messages

- The acceptors receives accept(2, w) and send learn(2, w). So w is choosen.



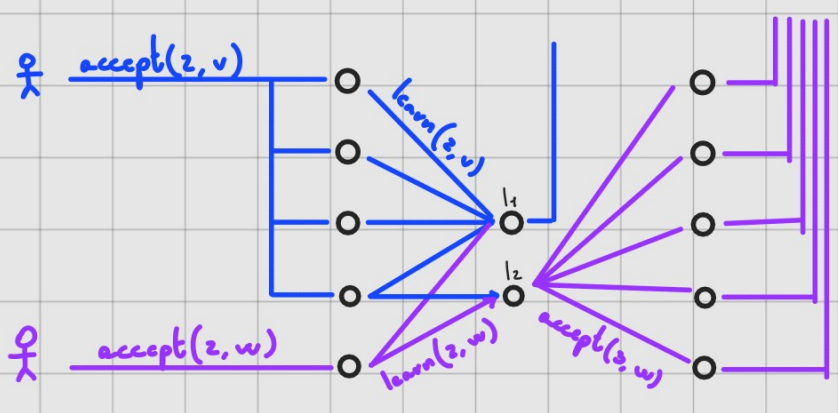
- In conclusion, both v and w are choosen ⇒ the protocol is not safe

Exercise 20: Assume that all proposers are learners as well. Let even rounds be assigned to proposers with the rules that we know. Moreover, If round 2i is assigned to proposer p, then also round 2i+1 is assigned to proposer p. Odd rounds are "recovery" rounds. If round 2i is a fast round and if the proposer of round 2i sees a conflict (it is also a learner), then the proposer immediately sends an accept for round 2i+1 with the value that has been most voted in round 2i, without any prepare and any promise. Is safety violated? If yes, show an example. If not, demonstrate safety.

Yes, the safety is violated. Let's assume a system with 5 nodes. The fast quorum is 4 while the classical quorum is 3.

- In round 2, since it is a FAST ROUND, it is possible that two different clients send value v and w directly to the acceptors
- Let's assume that only one node votes for w. All the other nodes vote for v. Consequently, we have a FAST QUORUM on v ⇒ v is choosen.
- A learner l_1 receives all the votes and learn v. Another learner l_2, instead, only receives 1 vote for v and 1 vote for w. So, l_2 cannot tell which is the most voted value in round 2 and simply picks w randomly.
- According to the premises, l_2 sends immidiately accept(3, w) to the acceptors. Then, they reply with learn(3, w) ⇒ w is learned

In conclusion, both v and w are learned ⇒ NOT SAFE



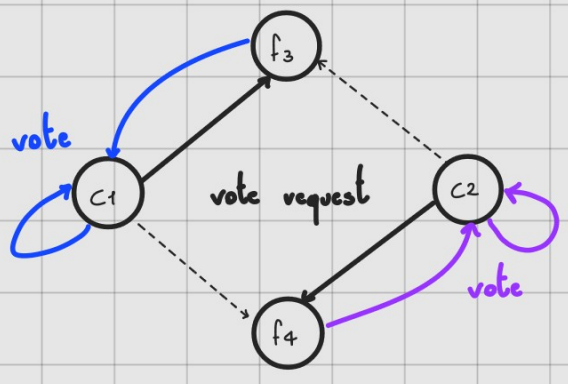
Exercise 21: You are an optimization freak. You realize that in Fast Paxos, in some cases, it is not necessary that the proposer collects n-f' (the Fast Paxos quorum) promises to take a decision. Which is the minimum quorum and under what hypothesis this minimum quorum is enough to take a decision?

Exercise 22: Show that Raft is not live.

Let's assume a system of 4 nodes: c1, c2, f3, f4. Nodes c1 and c2 become candidates in the same term (e.g., they had the same election timeout). Both vote for themselves and send the vote request to the others:

- f3 receives c1's vote request first and votes for it
- f4 receives c2's vote request first and votes for it

At this point we have a split vote. When the election timeout expires, both increment their term and start a new election. If the messages keep arriving in the same order the split vote repeats in every term and no leader will ever be elected.



Exercise 23: In Raft, it is sometimes possible that the elected leader for term t has not all the log entries that are stored in the followers. Show that, in that case, the log entries missing at the leader are actually not committed and so they can be overwritten by the new leader.

	1	2	3	4	5
S1	1_{e1}^{*}	1_{e2}^{*}	1_{e3}^{*}	CRASH	
S2	1_{e1}^{*}	1_{e2}^{*}	1_{e3}		
S3	1_{e1}^{*}	1_{e2}	LOST		
S4	1_{e1}^{*}	1_{e2}	LOST		
S5	1_{e1}^{*}	1_{e2}	LOST		

At this point, since each node can't vote for nodes with a smaller log:

- S2 can only vote for itself
- S3 can vote for S2, S3, S4, S5
- S4 can vote for S2, S3, S4, S5
- S5 can vote for S2, S3, S4, S5

Let's assume that they all vote for S4, except S2 which can only vote for itself.

	1	2	3	4	5
S1	l_{e1}*	l_{e2}*	l_{e3}*	CRASH	
S2	l_{e1}*	l_{e2}*	l_{e3}		
S3	l_{e1}*	l_{e2}	LOST		
S4	l_{e1}*	l_{e2}	LOST		
S5	l_{e1}*	l_{e2}	LOST		

Consequently, S4 becomes the leader (even if it has a smaller log than S2).
At this point, we are also sure that all the entries missing in S4 were actually not committed.

Dem. Let x be a log entry stored on some follower but missing from the leader S_4. Since the voters can't vote for nodes with a smaller log, if a majority voted for S_4 means that the majority of nodes don't have the entry x stored.

Consequently, since any entry requires a majority to be committed, x cannot be committed